

Benchmarking LLM-Based Agents for GitHub Issue Enhancement

Pouya Fathollahzadeh, *Student Member, IEEE*, and Ying Zou, *Member, IEEE*

Abstract—GitHub issues serve as the primary communication channel between users and developers for reporting bugs, requesting features, and proposing refactoring tasks. Issue descriptions are frequently vague, incomplete, or lacking essential technical details, which hinders both human developers and automated issue-solving agents. Large Language Model (LLM)-based agents offer the potential to automatically enhance issue descriptions by adding context, clarifying ambiguities, and structuring information. If agents solve issues from text, better text should mean better fixes. We test this assumption directly and at scale. We construct a SWE-bench-Live-style benchmark of 3,285 issue instances from 613 popular Python repositories and, from it, a validated execution subset of 382 issues, each with a buildable Docker environment and a gold patch. On this subset we run a full enhancer $\times$ solver matrix: coding agents used as *enhancers* (OpenHands, SWE-agent, Aider), a reward-gated selective enhancer (CL-Enhanced), and a no-enhancement baseline, against the same agents used as *solvers*, all driven by a single open-weight model (Qwen3-32B) to isolate the enhancement variable. We evaluate not “did the patch change” but “does the patch pass the repository’s tests”, using a gold-patch-gated, multi-method evaluability protocol that yields 279 of 382 issues (73.0%) as trustworthy to evaluate. Our results show that: (1) across the controlled agentic enhancer $\times$ solver matrix, issue enhancement produces no statistically significant improvement in fix correctness (paired McNemar exact test, all $p \geq 0.489$), and a zero-shot prompt enhancer likewise never helps and tends to hurt; the one promising small-sample signal weakened rather than strengthened as the sample grew, the signature of a non-effect; solver choice, not the wording of the report, dominates outcome (Aider 44% $\gg$ OpenHands 22% $\gg$ SWE-agent 6%). (2) No feature, across an 81-feature SE representation plus 500 TF-IDF terms, predicts a better enhancement under either an intrinsic (reward-model) or a downstream definition of success (all models at chance, ROC-AUC ≈ 0.49 – 0.56); strikingly, enhancement does raise the intrinsic quality score (57% of reports improve) yet yields no downstream correctness gain, so text-quality improvements do not transfer to program repair. (3) Open-coding 90 enhancements shows what the agents actually do: 91% impose a Problem/Reproduction/Expected template, while a root-cause diagnosis appears in only 10% and half lose content the reporter supplied. No rewrite move is associated with whether the fix improves (all $p \geq 0.227$). The null survives the two design objections that could explain it away. Constraining enhancement to be append-only, so that the reporter’s text always survives and only added context varies, leaves the matrix unchanged. Re-deriving the fix criterion from execution, after we found the shipped fail-to-pass labels had been parsed from diffs rather than observed, also leaves it unchanged: across 16 rescored conditions and 738 paired trials no comparison reaches significance. Throughout, enhancement raises the weaker solver and lowers the stronger one by amounts a resample of the solver predicts. We release the benchmark, the evaluation pipeline, and

all experimental artifacts.

Index Terms—Software Engineering, GitHub Issues, Issue Enhancement, LLM-Based Agents, SWE-bench, Automated Program Repair, Benchmark, Negative Results.

I. INTRODUCTION

GitHub issues are the primary mechanism through which developers and users communicate about software defects, feature requests, and maintenance tasks [2], [1]. A well-written issue report contains a clear problem description, reproduction steps, expected versus actual behavior, and relevant technical context such as version information, stack traces, or code snippets. When these elements are present, both human developers and automated solving agents can effectively diagnose and resolve the underlying problem.

In practice, however, issue descriptions are frequently incomplete, vague, or poorly structured. Prior work on bug report quality has shown that missing reproduction steps, ambiguous descriptions, and insufficient context are among the most common reasons for delayed or failed issue resolution [7], [8]. This quality gap is particularly consequential for automated issue-solving systems, which rely on the issue description as their sole input for generating code patches [1], [3].

The emergence of LLM-based coding agents such as OpenHands [4], SWE-agent [3], Aider [5], and AutoCodeRover [6] has opened a new possibility: using these agents not only to solve issues but also to enhance issue descriptions before solving. An enhancement agent can analyze the codebase, identify relevant source files, extract failing test information, and restructure the issue description to provide a clearer specification. If effective, this pre-processing step could improve the success rate of downstream solving agents. This hypothesis motivates a growing line of issue-enhancement tools, including reward-gated report rewriting. Yet enhancement is almost always evaluated intrinsically, by asking whether the rewritten report scores higher on a text-quality metric, and rarely extrinsically: does a better report actually cause a better downstream fix.

This paper closes that gap. Existing benchmarks such as SWE-bench [1] and SWE-bench-Live [2] focus exclusively on issue solving, treating the issue description as a fixed input. We instead treat the issue text as a manipulable stage. Our approach is built on the *solving-as-evaluation* paradigm: we measure the quality of an enhancement not through subjective text ratings but through its downstream effect on solver per-

formance. Formally, for an enhancement agent E and a solver agent S , the enhancement value is:

$$\Delta_{\text{resolved}} = \text{Resolved}(S, E(\text{issue})) - \text{Resolved}(S, \text{issue}) \quad (1)$$

where a positive Δ_{resolved} indicates that enhancement improved solving performance. This indirect evaluation provides an objective, reproducible measure of enhancement quality that is grounded in functional correctness.

Our study analyzes GitHub issue enhancement along three research questions, each with its own method:

RQ1: What is the ability of generative AI in enhancing GitHub issue reports? We benchmark an enhancer–solver agentic workflow: each enhancer rewrites the report, each solver then attempts a patch, and we measure enhancement ability by whether the resulting patch passes the repository’s tests. We evaluate a no-enhancement baseline plus enhancers of increasing sophistication across three solver agents, holding the model fixed.

RQ2: What features are related to a better enhancement? We fit a logistic regression over textual, structural, and enhancement-derived features to identify which are associated with a successful enhancement outcome, under both an intrinsic (reward-model) and an extrinsic (downstream-fix) definition of success.

RQ3: How are GitHub issue reports enhanced by generative-AI agents? We perform a qualitative analysis of the agent-generated reports to characterize recurring enhancement patterns and the failure modes that explain the RQ1 result.

The main contributions of this paper are as follows:

- 1) We construct a large-scale dataset of 3,285 GitHub issue instances from 613 repositories, and a validated execution subset of 382 instances each with a buildable environment and gold patch, suitable for the extrinsic evaluation of issue enhancement.
- 2) We propose and apply the *solving-as-evaluation* framework, measuring enhancement quality through downstream solver performance under a controlled model rather than subjective text assessment.
- 3) We provide a large-scale extrinsic benchmark (RQ1) whose finding is a rigorous negative result: no enhancement condition reliably improves fix correctness at any tested scale, and the one small-sample signal is traced to sampling noise by replication.
- 4) We provide a feature-level (RQ2) and qualitative (RQ3) account of which report/enhancement properties and rewrite behaviors relate to enhancement outcomes, surfacing an intrinsic-improves / extrinsic-flat gap.
- 5) We contribute an evaluation methodology that we argue is a prerequisite for trustworthy agent-repair measurement: a gold-patch-gated, three-method evaluability gate (73.0% coverage, 279/382), an artifact-recovery discipline, and a procedure for deriving fail-to-pass labels by execution rather than by parsing the test patch. The last matters beyond this study: the shipped labels claim a fail-to-pass test for 225 of 279 instances where execution finds one in 85, and correcting them removes every credited solve from one of the three solvers.

- 6) We release the benchmark dataset, evaluation pipeline, and all experimental artifacts in our replication package [31].

The rest of the paper is organized as follows: Section II discusses background and related work. Section III presents our methodology. Section IV presents the results for each research question. Section V discusses implications. Section VI discusses threats to validity. Section VII concludes.

II. BACKGROUND AND RELATED WORK

A. Issue Report Quality

What makes a bug report useful has been studied for nearly two decades. Bettenburg et al. [7] surveyed developers and found that steps to reproduce, stack traces and test cases are the most valuable elements, and also the most often missing. Zimmermann et al. [8] identified incomplete information as the primary barrier to fixing, with a large fraction of reports lacking sufficient detail. These studies establish the premise this paper tests: if reports are deficient in ways developers can name, then repairing those deficiencies should help whoever fixes the bug.

Earlier automated work attacked the problem indirectly. Duplicate detection [9], [10] reduces redundant reports, automated triage [11], [12] routes them to the right developer, and augmentation with relevant source code [13], [14] adds context a reporter omitted. All of it is validated intrinsically, against a text or model score, or against a proxy such as retrieval rank. The same pattern holds one level up, in requirements engineering, where Femmer et al. [22] detect *requirements smells* with lightweight linguistic checks and validate them against expert judgement rather than against project outcomes.

Language models have revived the idea of rewriting the report itself. Acharya and Ginde [19] generate bug reports with fine-tuned LLMs and evaluate them on CTQRS, ROUGE, METEOR and SBERT together with field-level F1, reporting that instruction-tuned models generalise to projects without training data. Al Fahim et al. [20] come closest to our setting: they enhance crash reports with fault locations and root-cause explanations, and compare a single-shot rewriter against an iterative agent that explores the repository, which is the same contrast we draw in Section V-F. Their evaluation is similarity to the developer’s patch rather than whether a patch produced from the enhanced report is correct.

This is the gap we close. Prior work establishes that LLMs can raise the measured quality of a report. None of it establishes that a higher-quality report yields a better fix, because none of it executes the resulting patch against the repository’s tests. Our contribution is the extrinsic measurement, and our finding is that the intrinsic gain does not transfer.

B. Issue-Solving Benchmarks

SWE-bench [1] introduced the evaluation instrument we reuse. Given a codebase and an issue description, a model generates a patch that is judged by unit tests in two categories: *fail-to-pass* (F2P) tests that must change from failing to passing, and *pass-to-pass* (P2P) tests that must keep passing

so that no regression is introduced. The design has since been extended along several axes. SWE-bench Verified [16] is a 500-instance subset filtered by expert annotators. SWE-bench-Live [2] addresses staleness with an automatically updatable benchmark drawn from recent issues, which is the design our own collection follows and the reason we restrict to issues created after the models' training cutoffs. Zhou et al. [25] quantify why that matters, measuring leakage across 83 software engineering benchmarks and finding it modest on average but severe on particular datasets. SWE-Bench Pro [17] targets long-horizon tasks across 41 repositories and uses copyleft and private code to resist contamination. Jiang et al. [18] survey 242 studies in this area and organise the benchmarks, scaffolds and training strategies that have accumulated around the task.

Every one of these evaluates the fix while treating the issue text as fixed input. We invert the instrument: the solver becomes a measuring device, and the question is whether changing the text changes the fix.

C. The Validity of Test-Based Judgement

Because our result depends on a test-based oracle, the known weaknesses of that oracle bear directly on it. Aleithan et al. [15] audited SWE-bench and found that 32.67% of successfully resolved instances contained the solution or a strong hint in the issue text, and that 31.08% of passing patches pass only because the tests are too weak to reject them; correcting for both drops one reported resolution rate from 12.47% to 3.97%. The annotation behind SWE-bench Verified [16] filtered out 68.3% of candidate instances for underspecification, unfair tests, or other defects.

This is a specific case of a problem the repair community has studied for years. Test-based program repair generates *plausible* patches that satisfy the available tests without being correct, and Fei et al. [23] survey the resulting literature on assessing patch correctness when the test suite cannot settle it. Strengthening the suite with generated tests is no free fix either: Schäfer et al. [24] find that a substantial fraction of LLM-written unit tests are invalid, and that search-based generation still achieves markedly higher coverage.

Two consequences shape this study. First, a test-based verdict is only as trustworthy as the tests behind it, which is why we gate every instance on a gold-patch probe and, in Section V-H, re-derive the fail-to-pass labels by execution after finding that the shipped labels had been parsed from the test patch rather than observed. Second, the filtering that makes these benchmarks trustworthy also selects for issues that are already actionable, since an instance survives only if a real fix exists and its tests discriminate. That selection is a threat to our own external validity and we return to it in Section VI.

D. LLM-Based Coding Agents

Coding agents wrap a language model in a loop with tools for reading, editing and executing code. SWE-agent [3] contributes an agent-computer interface designed for software engineering tasks. OpenHands [4] provides a general platform for software development agents. Aider [5] is a command-line pair-programming tool. AutoCodeRover [6] performs

autonomous program improvement through structured code search. Two results in this line bear on our hypothesis that enhancement helps by supplying localisation. Xia et al. [26] show that a pipeline with an explicit localise-then-repair decomposition and no agent loop is competitive with autonomous agents, which implies localisation is a separable and consequential step. Ma et al. [27] improve issue resolution specifically by giving the agent a structured view of the whole repository to explore. We use these systems in both roles the paper requires, as solvers and, with the same tool loop and the same repository access, as enhancers.

E. Issue Enhancement, and Why It Might Not Help

Issue enhancement means automatically improving a report by adding context, clarifying ambiguity, restructuring content or appending diagnostic information. Unlike solving, it produces text rather than a patch, and that text can be consumed by a human developer or by an automated solver.

One line of evidence suggests it should help: reports are demonstrably deficient [7], [8] and models can demonstrably improve them on quality metrics [19]. Another suggests caution. Language models are sensitive to prompt wording in ways that are not tied to information content, and Paleyes et al. [21] show that meaning-preserving variation in a prompt changes the functionality and quality of generated code. If a solver is sensitive to how a report is phrased and not only to what it contains, then rewriting can move outcomes in either direction without adding anything. Distinguishing these two accounts requires an extrinsic, executed measurement on a sample large enough to detect a directionless perturbation, which is what we report.

III. METHODOLOGY

Fig. 1 presents an overview of our approach, which consists of two components: (1) a data collection and dataset construction pipeline, and (2) an enhancement–solving–evaluation workflow.

A. Data Collection and Dataset Construction

We construct a SWE-bench-style benchmark from recent GitHub issues to ensure evaluation data postdates the training cutoffs of the LLMs under study. Fig. 2 illustrates the pipeline.

1) *Stage 1: Repository Selection*: We fetch GitHub repositories with more than 1,000 stars, yielding 10,609 repositories, and apply three quality filters: (i) **language**: Python constitutes more than 60% of the codebase; (ii) **fork activity**: more than 200 forks; and (iii) **issue and PR volume**: the combined count exceeds 200. After these filters, 3,621 repositories with 10,463 associated issues remain.

2) *Stage 2: Temporal Filtering*: To ensure recency and avoid training-data contamination, only issues created after August 1, 2025 are retained. This reduces the dataset to 2,748 repositories with 7,714 issues.

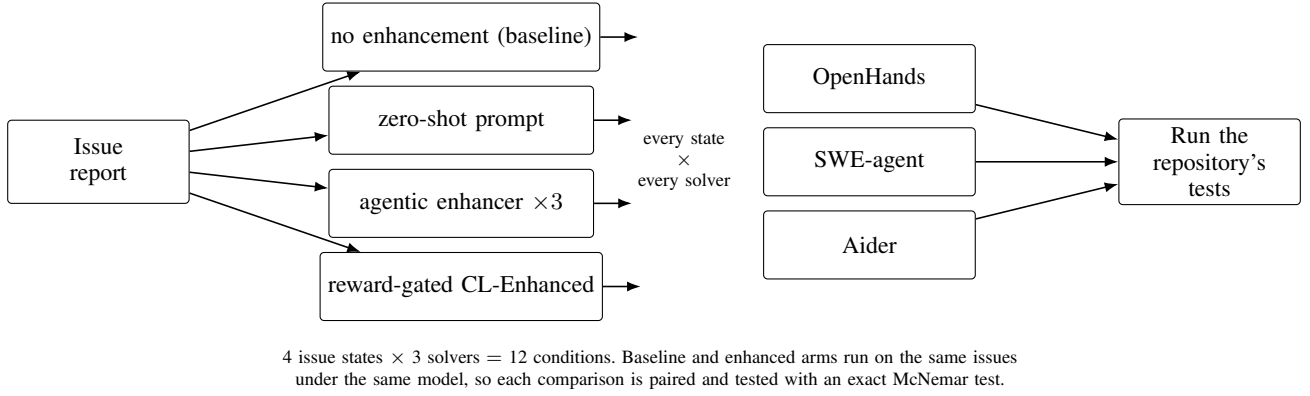


Fig. 1. Overview of our approach. Original issues are processed by each enhancement condition (including a no-enhancement baseline). Each enhanced issue is then solved by three independent solver agents. Enhancement is measured by the change in the solver’s resolved rate, scored first on no-regression and, in Section V-H, on an executed fix criterion.

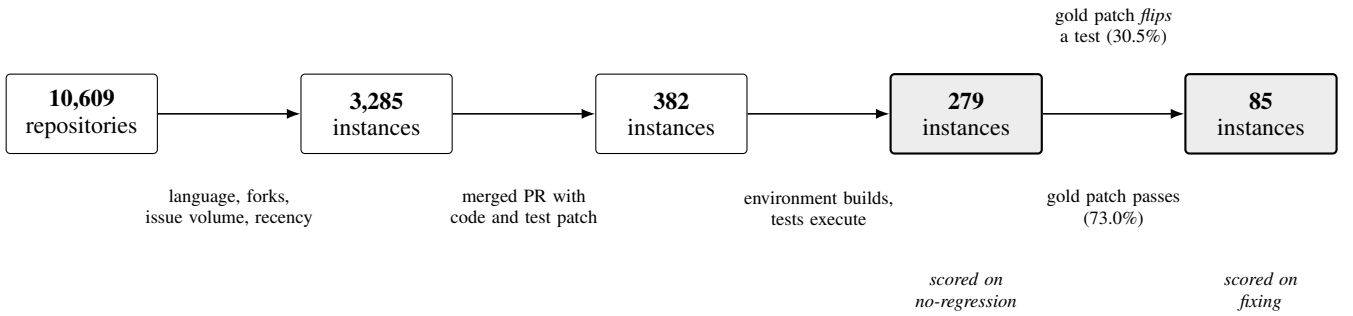


Fig. 2. Data collection and the two evaluability gates (shaded). Successive filters for language, activity, recency and merge-based linking yield 3,285 benchmark-ready instances across 613 repositories, and validated environment construction yields the 382-instance execution subset. The first gate keeps an instance only if the gold patch itself passes under one of three test-command methods, which is what makes a no-regression score trustworthy. The second, added after we found the shipped fail-to-pass labels had been parsed from diffs rather than executed, keeps only instances where the gold patch demonstrably turns a failing test green. Every condition is reported on the 279; Section V-H rescores them on the 85.

3) *Stage 3: Merge-Based Linking and Test Coverage:* We retain only issues that (a) were closed by a merged pull request (establishing a ground-truth fix); (b) whose PR includes a non-empty code patch and a non-empty test patch; and (c) that have at least one pass-to-pass test ($P2P > 0$). This yields 613 repositories with 3,285 issues.

4) *Stage 4: Validated Execution Environments:* For the extrinsic, execution-based evaluation, each issue must have a reproducible environment in which tests actually run. We build a per-issue Docker image and verify that the repository’s designated tests execute and that the gold patch is applicable. The instances that pass this validation on our primary compute node form the 382-issue execution subset used for the enhancer $\times$ solver matrix. Each such instance carries a buildable image, a recorded test command, and a gold patch.

B. Issue Enhancement Conditions

We evaluate a no-enhancement baseline against a ladder of enhancers of increasing sophistication, so that scale of machinery is controlled:

- **No Enhancement (Baseline):** the original issue is passed directly to the solver. Control condition.
- **Zero-Shot (Raw LLM):** a single LLM call restructures the issue without agent-level tool use, measuring the value of prompt-only rewriting.

- **Ready-to-use agents: OpenHands** [4], **SWE-agent** [3], and **Aider** [5], each given the issue text and instructed to enhance the description, rewriting *unconditionally*. These operate on the report *without* a checkout of the repository under study: each runs in an empty working directory containing only the issue, so any file reference it emits is inferred from the report rather than read from the code. Their loop budgets differ by tool: OpenHands runs 8 steps, SWE-agent 10, and Aider a single non-interactive turn. Section V-F reports a separate enhancer built to remove exactly this limitation.
- **CL-Enhanced (reward-gated):** a managed enhancer that *gates* on a learned reward model and only rewrites reports it judges low-quality, using retrieval-augmented (RAG) grounding. This is a *selective* enhancer (Section V-C).

C. Solver Agents

To evaluate enhancement through downstream solving impact, we use three independent solver agents: OpenHands [4], SWE-agent [3], and Aider [5], each in solving mode. Using multiple independent solvers reduces the risk that enhancement effects are artifacts of a single solver; an enhancement that helps across solvers provides stronger evidence of genuine quality improvement.

Controlled model. Every enhancer and solver is driven by a single open-weight model, Qwen3-32B, served from a private inference endpoint at temperature 0.0. Holding the model fixed isolates the *enhancement method* as the only variable, so any difference is attributable to the rewriting, not to model capability. Enhancement and solving execute in isolated Docker environments to prevent cross-contamination.

D. Evaluation Framework

Our evaluation uses the SWE-bench-Live harness [2], [1] to assess solver-generated patches against ground-truth tests.

1) *Resolved Rate (P2P)*: We score a patch as *resolved* if, after applying it (plus the issue’s test patch) inside the validated image, the repository’s designated P2P tests execute and pass with no failures:

$$\text{Resolved} = \begin{cases} 1 & \text{if all P2P tests pass} \wedge |\text{P2P}_{\text{observed}}| > 0 \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

We report P2P-resolved as the primary criterion because the F2P labels shipped with the instances were parsed from the test patch’s diff rather than observed. Every instance carries `offline_test_patch_diff_parse` as its derivation method, and the labels do not survive contact with execution: they claim a fail-to-pass test for 225 of 279 instances where running the tests finds one in 85, and the two sets agree on 45. P2P-resolved is a no-regression signal, augmented by the applied test patch; we treat it as directional. Section V-H re-derives both label sets from execution and rescores every condition under the stricter criterion.

2) *The Evaluability Gate*: Not every issue can be trusted to evaluate. We gate on a gold-patch probe: an instance is *evaluable* only if the gold patch itself passes under the chosen method. Because a single generic test command fails on many repositories (framework-specific runners, stale node-IDs), we combine three methods per instance and keep whichever the gold patch validates: **v2**, the repository’s real validated test commands recovered from the build; **v3**, a label-agnostic file-level run (execute the test files, require ≥ 1 passed and 0 failed, which is robust to mislabeled node-IDs); and **v1**, a generic fallback. This lifted evaluable coverage to 73.0% overall (279/382), and up to 75.8% on the largest tranche, versus $\sim 30\%$ for a single generic method.

3) *Enhancement Coverage and Delta*: *Coverage* is the proportion of issues an enhancer actually modifies, $\text{Coverage} = |\{i : E(i) \neq i\}|/|I|$. The primary comparison metric is the resolved-count *delta* against the baseline (Eq. 1); a positive delta indicates net improvement, a negative delta indicates that enhancement harmed solving.

4) *Statistics*: For each enhancer-vs-baseline comparison on a given solver, outcomes are paired (same issues, with vs. without enhancement), so we use the exact McNemar test on the discordant pairs and report the pair counts. We report Wilson 95% confidence intervals on each resolved rate. Effect sizes are absolute differences in resolved count over the fixed evaluable denominator (empty or failed patches count as unresolved). Solver agents are randomised procedures, so a

single run per condition would confound the effect with run-to-run variance; following the guidance of Arcuri and Briand [28] on randomised algorithms we pair every comparison on the same issues and calibrate each result against a measured resample rate rather than against an assumed zero.

IV. RESULTS

A. RQ1: What Is the Ability of Generative AI in Enhancing Issue Reports?

1) *The correctness matrix: no enhancer improves any solver*: All twelve cells of Table I and all nine McNemar *p*-values were recomputed from the stored solver patches in a separate pass and reproduce the reported values exactly. Table I gives the final correctness matrix at $n = 382$ (279 gold-evaluable issues), with each enhancer compared to the baseline for each solver. **Among the three agentic enhancers, which form the controlled comparison and are all built on the same ~ 30 -step agent loop, no enhancer $\times$ solver comparison reaches significance; every $p \geq 0.489$, and the largest absolute effect is 7 issues out of 279.** The one comparison that could be called a “best case”, Aider-enhancement into the OpenHands solver (+7), is nowhere near significance and is not the pairing any prior signal predicted. The crudest enhancer, a single zero-shot prompt (Raw LLM), does not help either and if anything hurts: it leaves OpenHands unchanged (+1, $p = 1.0$) but lowers Aider by 12 (−12, $p = 0.213$) and SWE-agent by 6, the latter’s nominal $p = 0.031$ being a non-determinism artifact on the error-prone SWE-agent solver rather than a real effect (Table I, footnote). That the least sophisticated enhancer trends negative, not positive, is consistent with the study’s arc: added text perturbs the solver without adding solver-actionable signal.

2) *The one promising signal was sampling noise*: At $n = 100$ the headline pairing (OpenHands-enhancement $\rightarrow$ Aider solver) looked promising: +4 resolved (30 $\rightarrow$ 34), $p = 0.388$. Replication kills it (Table II): the effect shrank toward zero and the *p*-value rose toward 1 as data accumulated, the diagnostic of a non-effect rather than of insufficient power. A genuine effect of fixed size sharpens under more data; this did the opposite. The same pattern held for the OpenHands solver, where an $n = 74$ “+6” reversed to −3 on a fresh 100 issues before settling near null. Had we stopped at $n = 100$, the paper would have carried a false positive.

3) *Solver choice dominates; enhancement perturbs without direction*: Two robust facts survive at every scale. **(1) Solver capability is the dominant factor**: baseline resolved rates are Aider 44% $\gg$ OpenHands 22% $\gg$ SWE-agent 6%, stable across all three tranches and far larger than any enhancement effect. The choice of *who fixes* matters an order of magnitude more than *how the report is worded*. **(2) Enhancement is active but directionless**: for the two responsive solvers, 65–78 issues per comparison change outcome (discordant pairs), 23–28% of the evaluable set, but the flips are balanced (helps $\approx$ hurts), so there is no net gain. A model-fit note reinforces the point: the SWE-agent solver is effectively inert to enhancement (0–1 discordant pairs) because, with Qwen3-32B, the large majority of its runs terminate in an agent-protocol error before

TABLE I

RQ1 CORRECTNESS MATRIX AT $n = 382$: RESOLVED / 279 EVALUABLE (P2P), WITH Δ VS. BASELINE AND McNEMAR EXACT p . ROWS ARE SOLVERS, COLUMNS ENHANCERS. THE THREE AGENTIC ENHANCERS FORM THE CONTROLLED COMPARISON (IDENTICAL ~ 30 -STEP LOOP, MODEL HELD FIXED); THE ZERO-SHOT (RAW LLM) COLUMN IS A SINGLE-PROMPT REFERENCE.

Solver (baseline)	enh: OpenHands	enh: SWE-agent	enh: Aider	enh: Raw LLM (0-shot)
OpenHands (60, 22%)	58 (-2, .910)	66 (+6, .561)	67 (+7, .489)	61 (+1, 1.0)
SWE-agent (18, 6%)	18 (0, 1.0)	19 (+1, 1.0)	19 (+1, 1.0)	12 (-6, .031 [†])
Aider (124, 44%)	125 (+1, 1.0)	119 (-5, .649)	124 (0, 1.0)	112 (-12, .213)

[†]The only $p < .05$ cell in the matrix; a non-determinism artifact on the error-prone SWE-agent solver (6 discordant pairs, all one-directional, on a solver that fails $\sim 86\%$ of runs on the agent-computer interface), not an enhancement effect, exactly parallel to the CL-Enhanced SWE-agent case in Section V-C.

TABLE II

OPENHANDS-ENHANCEMENT $\rightarrow$ AIDER SOLVER, ACROSS SCALES.

Scale	Resolved (base $\rightarrow$ enh)	Δ	McNemar p
$n = 100$ (74 eval)	30 $\rightarrow$ 34	+4	0.388
$n = 200$ (141 eval)	64 $\rightarrow$ 66	+2	0.856
$n = 382$ (279 eval)	124 $\rightarrow$ 125	+1	1.000

any patch is produced ($\sim 86\%$ in a 100-issue audit), a failure of the model-harness interface rather than of the issue text.

The per-solver direction of the effect is itself informative. Pooling every 40-issue replication cell, enhancement raises the weaker solver by 26 resolved issues (OpenHands, baseline rate 0.21) and lowers the stronger one by 31 (Aider, baseline rate 0.47), while both margins sit within a rescue or two of what a resample predicts: 78 rescues against 75.2 expected and 52 breakages against 41.6 for OpenHands, 46 against 50.1 and 77 against 93.0 for Aider. Enhancement lifts whichever arm has more room to rise and depresses whichever has more to lose, which is regression to the mean rather than added information. This also accounts for the individual cells that looked encouraging in smaller samples: they fall on the low-baseline solver, where a resample can move the count in only one direction.

RQ1 Summary. Across the controlled agentic enhancer $\times$ solver matrix on 279 evaluable issues, no enhancer improves fix correctness for any solver (all McNemar $p \geq 0.489$); the crudest zero-shot prompt enhancer only ever matches or hurts. The one small-sample signal regressed toward zero under replication, a non-effect. Solver capability (Aider 44% $\gg$ OpenHands 22% $\gg$ SWE-agent 6%) dominates, and enhancement perturbs a quarter-to-a-third of outcomes without net direction.

B. RQ2: What Features Are Related to a Better Enhancement?

Approach. From each issue’s title and body we extract the full **81-feature SE representation** of prior reward-model work, spanning size, readability (Flesch, Flesch-Kincaid, ARI), part-of-speech counts, structural markup (fenced/inline code, URLs, images, checklists, headers, lists, tables), content flags (stack trace, error message, logs, reproduction steps, expected/actual behavior, environment info), and lexical statistics. We use it two ways: *descriptively*, tallying which features an enhancement adds; and *predictively*, regressing a success outcome on the features under two operationalizations: an **extrinsic** label (the enhanced report yields a downstream

TABLE III

TOP SE FEATURES *added* BY ENHANCEMENT (ABSENT IN THE ORIGINAL, PRESENT AFTER; OPENHANDS, $N = 304$ TRULY-ENHANCED).

Feature added	Issues gained	% of enhanced
reproduction steps	259	85%
expected behavior	138	45%
list items	135	44%
actual behavior	124	41%
section headers	121	40%
logs	66	22%
inline code	60	20%
environment info	26	9%
links	15	5%
checkboxes	12	4%

resolved patch), over the 81 SE features; and an **intrinsic** label (the learned reward model scores the enhanced report ≥ 0.5), over the full **581-feature** set (81 SE + 500 TF-IDF terms). Both use L1-regularized logistic regression tuned by 5-fold cross-validation.

1) *What enhancement adds:* Comparing SE features before and after enhancement (OpenHands, over its 304 truly-enhanced issues), the additions are a uniform template: the agent rewrites almost every report into a structured Problem / Reproduction / Expected / Actual form (Table III). These are the fields the reward model rewards, which explains the intrinsic gain below; but several of them, reproduction steps and expected/actual behavior, are facts the agent cannot know and often invents, foreshadowing why they do not help the solver.

2) *No feature predicts a better enhancement:* **Extrinsic view.** The 81-feature model reaches 5-fold cross-validated ROC-AUC = 0.55, indistinguishable from the 0.50 no-signal line. L1 retains 14 features with modest odds ratios (Table IV); the strongest are informativeness properties of the original report (bodies with logs, OR 1.36; a stack trace, OR 1.20), not of the rewrite. On the 65 issues where enhancement changed the Aider outcome, it helped 33 and hurt 32; predicting helped-vs-hurt yields ROC-AUC ≈ 0.58 at $n = 65$, no reliable signal.

Intrinsic view: the key contrast. With the full 581-feature representation and the reward model, enhancement is *not* inert: original score mean 0.473 $\rightarrow$ enhanced 0.578; 218/382 (57%) of reports improve and 122 (32%) cross from low- to high-quality. Yet which issue gets a successful enhancement is still unpredictable from its 581 features (AUC = 0.49, chance). The agent enhances uniformly, turning almost any report into a similarly structured, higher-scoring one, so no input feature

TABLE IV

L1-SELECTED SE FEATURES FOR A RESOLVED FIX (BEST CELL, $n = 279$; FULL 81-FEATURE MODEL, STANDARDIZED OR). REMAINING SELECTED FEATURES HAVE $|OR - 1| < 0.05$; MODEL AUC = 0.55.

Feature (from title/body)	Standardized OR
body_has_logs	1.36
body_has_stack_trace	1.20
body_has_environment_info	1.09
body_flesch_reading_ease	1.03
title_contains_feature	0.87
num_urls_in_body	0.87
body_avg_word_length	0.88
title_ends_with_punctuation	0.89
title_stopword_ratio	0.90

TABLE V

RQ2: ALL THREE FEATURE MODELS LAND AT CHANCE.

Model	Features	Outcome	n	Pos.	CV AUC
extrinsic	81 SE	downstream resolved	279	45%	0.55
intrinsic	581	reward ≥ 0.5	382	69%	0.49
direction	81 SE	helped vs. hurt	65	51%	0.58

discriminates success. Table V summarizes: all three feature models land at chance.

RQ2 Summary. No feature, across the full 81-SE + 500-TF-IDF representation, is meaningfully associated with a better enhancement, on either an intrinsic or a downstream definition. The sharpest finding is the contrast: enhancement reliably raises the *intrinsic* quality score (57% improve, 32% cross threshold) but produces no downstream correctness gain and no feature-predictable subpopulation. Text-quality gains do not transfer to program repair.

C. RQ3: How Are Issue Reports Enhanced?

We open-coded a stratified sample of 90 enhancements, drawn with seed 42 as ten per enhancer $\times$ outcome cell over the three agentic enhancers and the three outcomes (helped, hurt, unchanged), with the solver held at OpenHands and the model at Qwen3-32B. The codebook has seven pattern codes and five failure-mode codes, both multi-label.

Coding procedure, and what our agreement figure does and does not mean. A large language model coded all 90 items, then coded them a second time with the first pass hidden and the rows reshuffled. Per-code agreement between the two passes is high: mean $\kappa = 0.90$ on rewrite moves and 0.96 on failure modes, with exact label-set matches on 76 and 88 of 90 items respectively.

That figure is a measure of how stable the coding procedure is when re-run. *It is not inter-rater reliability*, and we do not present it as such: both passes come from the same model, so they share whatever systematic misreadings that model has, and no amount of agreement between them would reveal one. A second human coder is the outstanding item for this research question. Two further limits bound what the pass can see. It cannot check a cited symbol against the repository, so the hallucination code is applied only where an assertion is not derivable from the original report, never where it is

TABLE VI

REWRITE MOVES AND FAILURE MODES OVER 90 CODED ENHANCEMENTS, MULTI-LABEL.

Rewrite move	%	Failure mode	%
Restructure into sections	91	Drops or buries content	50
Add reproduction steps	67	None observed	32
Add code context	50	Abstained	9
Add environment info	14	Hallucinated specifics	9
Add root-cause hypothesis	10	Over-specified	2
Surface a stack trace	6		
No substantive change	9		

merely unverified. And the two passes were shown excerpts of different lengths, which accounts for part of the residual disagreement: the second pass assigned the code-context and reproduction-step codes slightly more often (51 versus 45, and 70 versus 60) because more of each report was visible. We report the counts below from the first pass.

What the agents do. The rewriting is close to uniform.

Ninety-one percent of enhancements impose the Problem / Reproduction / Expected / Actual template, confirming at the level of individual reports what the feature deltas of Section IV show in aggregate. The moves that would carry genuine diagnostic content are the rare ones: a root-cause hypothesis appears in 10% and a surfaced stack trace in 6%. Half of all enhancements drop or bury information the reporter supplied, which is the same behaviour the append-only constraint of Section V-G was built to remove. The median length ratio is $1.06\times$, but that central tendency hides the real behaviour: 34 of 90 outputs are shorter than their input and 24 are more than twice as long, so “enhancement” is about as often compression as expansion.

No move is associated with helping. Cross-tabulating each code against outcome gives a flat table. Restructuring splits 26/28/28 across helped, hurt and unchanged; reproduction steps 21/18/21; code context 14/16/15. No pattern code and no failure-mode code reaches significance against outcome (χ^2 , all $p \geq 0.227$). The qualitative data therefore agree with RQ1 and RQ2: the agents apply a consistent template, and nothing about which template moves they apply predicts whether the fix improves.

A tooling artifact worth reporting. Ten of the 90 enhancements (11%) carry scaffolding from the enhancer’s own execution: unfilled template slots where a file path or error should be, the benchmark instance identifier pasted in as the issue title, a shell heredoc terminator, and in one case pip output and a `sudo` error string from the enhancing machine inserted as reproduction steps. These appear only in the OpenHands (6) and Aider (4) enhancers. They are a reminder that an agent writing prose into a file has no mechanism to check that the file is a well-formed report.

RQ3 Summary. Enhancement is a uniform restructuring into Problem/Reproduction/Expected/Actual sections, applied to 91% of reports, while root-cause diagnosis appears in only 10%. Half of all enhancements lose reporter-supplied content. No rewrite move is associated with whether the fix improves (all $p \geq 0.227$), which is the mechanism behind the directionless perturbation in RQ1.

V. DISCUSSION

A. Why Enhancement Fails to Transfer

The issues in an execution-gated repair benchmark are, by construction, *already actionable*, since they were selected because a solver could plausibly fix them. Rewriting an already-actionable report adds little signal and can just as easily perturb the solver off a working trajectory as onto one. Enhancement may help most exactly where these benchmarks contain the fewest examples: genuinely underspecified reports. This re-frames the “enhancement paradox”, that adding information does not help, as a selection effect of the benchmark rather than a universal law.

B. Measurement Discipline Was Decisive

Two classes of infrastructure artifact would each have manufactured a false result if left uncorrected, and both were caught by cross-checking rather than trusting raw counts: (i) **solver timeouts under concurrency**: slow iterative solvers, and even Aider on heavy repositories, hit wall-clock limits and produced empty patches that are *not* real failures; re-solving at lower concurrency recovered them (e.g., 20 of 27 Aider timeouts recovered at a median 881 s). (ii) **Permission-lost patches**: the runtime occasionally wrote its patch file unreadable to the host, silently zeroing $\sim 2\%$ of OpenHands results; these were recovered by a privileged read. A study scoring the raw output would have reported spuriously low, and unevenly distributed, enhancement effects.

C. The CL-Enhanced (Reward-Gated) Arm

The agents above rewrite unconditionally. Our managed enhancer, CL-Enhanced, instead gates on a learned reward model and only rewrites reports it judges low-quality. Its reward model judged 216 of 382 issues (57%) already adequate and abstained (returning the original), acting on the other 166 (43%) with RAG-grounded rewriting, making it a selective enhancer and a cleaner test of whether targeted enhancement transfers. The methodologically correct comparison is on the subset it chose to rewrite; on the **129 evaluable issues CL-Enhanced rewrote, no solver improves** (Table VII). A cautionary note: over all 279 evaluable, the SWE-agent comparison reads $\Delta -9$, $p = 0.004$, which is apparently significant. It is an artifact of non-determinism: 6 of those 9 “losses” fall on abstained issues whose text was identical to baseline, so they reflect run-to-run solver variance, not any effect of rewriting. The enhanced-only subset removes this confound and shows the true null. The takeaway completes the arc: **neither unconditional generic enhancement nor selective reward-gated enhancement improves downstream fix correctness.**

TABLE VII
CL-ENHANCED VS. BASELINE, ON THE 129 EVALUABLE ISSUES IT REWROTE (MCNEMAR EXACT).

Solver	Baseline $\rightarrow$ CL-Enhanced	Δ	p
OpenHands	32 $\rightarrow$ 32	0	1.00
SWE-agent	8 $\rightarrow$ 5	-3	0.25
Aider	58 $\rightarrow$ 55	-3	0.74

TABLE VIII
CROSS-MODEL ROBUSTNESS: ENH:AIDER $\rightarrow$ SOL:OPENHANDS ON THE 279 EVALUABLE, BASELINE VS. ENHANCED UNDER EACH MODEL (MCNEMAR EXACT, PAIRED WITHIN MODEL).

Model	Baseline	enh:Aider	Δ	p
Qwen3-32B (open)	60/279 (22%)	67/279 (24%)	+7	0.489
GPT-5-mini (frontier)	155/279 (56%)	159/279 (57%)	+4	0.712

D. Cross-model robustness: a frontier solver

Our controlled design holds the model fixed at an open-weight Qwen3-32B, which raises the question of whether the null is an artifact of a weak solver. To test this, we re-ran the strongest cell, enhancer Aider into the OpenHands solver, on the 279 evaluable issues with a stronger, function-calling-native commercial model (GPT-5-mini) for both the baseline and the enhanced condition (a valid paired comparison, since both share the model). Two things change and one does not (Table VIII). The absolute resolve rate rises sharply: OpenHands’ baseline goes from 22% (Qwen3-32B) to 56% (GPT-5-mini), a $2.6\times$ increase, confirming that solver/model capability, not report wording, is the dominant lever. But enhancement still does not help: 155 $\rightarrow$ 159 resolved ($\Delta +4$ of 279, McNemar exact $p = 0.712$), and the effect remains directionless (66 discordant pairs: 31 hurt, 35 helped). A $2.6\times$ more capable, tool-calling-native solver does not make enhancement transfer to correctness, and the negative result is robust across a weak open model and a strong frontier model.

E. Capability spread across many models

We broaden the two-model check to the entire local model inventory of our server as an illustrative probe over every model available on the shared Ollama endpoint. On a 20-issue sample drawn from the issues GPT-5-mini resolves at baseline, i.e., issues that are *provably solvable*, we run the same enh:Aider $\rightarrow$ sol:OpenHands cell (cap 30) with each model, plus two commercial references (Table IX). Two patterns are stark. First, capability is a cliff, not a slope: GPT-5-mini resolves all 20 (by construction), the strongest open model (Qwen3-32B) resolves only 6, and every other model, open or commercial, resolves 0–4. On issues known to be solvable, only the frontier model reliably closes them in the agent loop. Second, consistent with RQ1, enhancement helps none of them: the baseline-to-enhanced change ranges -4 to $+2$ across all nineteen models that load (two further server models would not load at all on the available Ollama/driver), uniformly negligible or negative. Code-specialized models (Qwen2.5-Coder, DeepSeek-Coder, Granite-Code) fare no better than

TABLE IX

FULL SERVER INVENTORY: ENH:AIDER $\rightarrow$ SOL:OPENHANDS ON 20 GPT-5-MINI-SOLVABLE ISSUES (CAP 30, P2P-RESOLVED), FOR EVERY MODEL ON THE LOCAL OLLAMA SERVER PLUS TWO COMMERCIAL REFERENCES (*). A 20-ISSUE, SINGLE-SEED SPOT-CHECK, *not* A POWERED COMPARISON; THE SAMPLE IS UPWARD-BIASED FOR GPT-5-MINI BY CONSTRUCTION. TWO SERVER MODELS WOULD NOT LOAD ON THE AVAILABLE OLLAMA/DRIVER.

Model	Baseline	enh:Aider	Δ
GPT-5-mini* (frontier)	20/20	18/20	-2
Qwen3-32B	6/20	5/20	-1
Llama-3.3-70B	4/20	0/20	-4
Qwen2.5-32B	2/20	3/20	+1
GPT-4.1-mini*	1/20	2/20	+1
Qwen3.5-35B	1/20	2/20	+1
DeepSeek-R1-70B	1/20	1/20	0
gpt-oss-120B (reasoning)	1/20	1/20	0
Llama-3-8B (fp16)	1/20	0/20	-1
GLM-4.7-flash-30B	0/20	2/20	+2
Qwen2.5-72B	0/20	1/20	+1
Qwen2.5-Coder-32B	0/20	1/20	+1
DeepSeek-Coder-33B	0/20	0/20	0
DeepSeek-Coder-v2-16B	0/20	0/20	0
DeepSeek-R1-8B	0/20	0/20	0
Granite-Code-34B	0/20	0/20	0
Llama-3-8B (q4)	0/20	0/20	0
Mixtral-8 $\times$ 22B	0/20	0/20	0
Mixtral-8 $\times$ 7B	0/20	0/20	0
gemma-3-12B	did not load		
Qwen3-Coder-30B	did not load		

general ones, which indicates that agentic competence rather than coding knowledge is the bottleneck.

A trajectory audit of this cell identifies the mechanism behind the cliff, and it is one of submission, not fix quality. GPT-5-mini emits a well-formed patch on 20/20 instances (median 1.2 KB, touching a single file); every other model emits *no patch at all* on 13–20 of the 20. The models do not lose a code-correctness contest. They never reach the point of proposing a fix, dying in the agent loop either by repeating an identical action (AgentStuckInLoopError; 17/20 for DeepSeek-Coder-v2-16B, 16/20 for Mixtral-8 $\times$ 22B) or by exhausting all 30 iterations without converging (14/20 for Qwen2.5-Coder-32B, 13/20 for GLM-4.7-flash). The failure mode “clean patch that does not fix the bug” is essentially absent from the data. Two controls localize the deficit further. It is *not* tool-calling support: GPT-4.1-mini is function-calling-native, like GPT-5-mini, yet still emits 19/20 empty patches and exhausts the iteration cap 10 times. And it is not code specialization: the three code-tuned models emit zero well-formed patches between them. What separates the frontier model is sustained agentic competence, keeping a long multi-step task coherent and terminating cleanly, which also makes it the fastest model in the set ($\sim$ 2 min median versus 3–25 min), because it converges rather than thrashes. This is a small, deliberately selection-biased spot-check (20 issues, chosen as GPT-5-mini-solvable, single seed) and *not* a powered comparison; we report it only to show that the RQ1 null and the solver-dominance finding persist across the full capability range of models available to us.

F. Does repository access rescue enhancement?

A natural objection to the RQ1 null is that our enhancers never see the code. They are text-to-text rewriters: each receives the issue title and body in an empty working directory, with no checkout of the repository under study. The solver, by contrast, runs in a per-instance container with the repository at the target commit under /testbed and a full tool loop over it. This asymmetry bounds what any such enhancer could achieve:

$$\text{enhancer output} = f(\text{issue text}), \quad \text{solver input} = \text{issue text} + \text{repository}$$

Anything the enhancer derives, the solver can derive from a strict superset by reading the code. A text-only enhancer therefore cannot add information to the solver’s input; it can only re-word what the solver already had.

To test whether removing that asymmetry changes the result, we built a repository-grounded enhancer: an agent given exactly what the solver gets, namely the same container, the same repository at /testbed, and the same 30-step tool loop, and asked to enrich the report from what it finds. It receives no oracle information: never the gold patch, the test patch, the hints field, or the fail-to-pass test names. Two constraints follow from our diagnostics. It is append-only: the original report must survive verbatim and may only be added to, which removes the compression failure we observed in the text-only enhancers (the OpenHands enhancer’s median output is shorter than its input). And every cited path is verified to exist in the container before the output is accepted, because unverified references measurably hurt.

The enhancer works as designed. On a 20-issue random sample with GPT-5-mini it enriched 17/20 reports, cited 50/50 file references that were verified to exist with zero invented paths, and expanded reports by a median factor of 5.96, in one case turning a 234-character issue into a 7.5 KB report with real functions and line numbers. Yet on correctness it changes almost nothing: 13/20 $\rightarrow$ 12/20 resolved (per-protocol, on the 17 treated issues, 11/17 $\rightarrow$ 10/17). Of the seven issues the baseline failed, enhancement rescued zero; against a pre-registered threshold of ≥ 4 rescues this is squarely within the null. Nineteen of twenty issues had an identical outcome in both arms.

This is the strongest form of the negative result: a well-formed, repository-grounded, verified, non-destructive enhancement rescued nothing, which is what the redundancy argument above predicts.

A secondary observation initially pointed the other way on harm. Enhancement broke only 1 of 13 working solves in this pilot, where a resample of the solver would break 5.3 ($p = 0.011$), and the same shape appeared in one cell of the append-only matrix of Section V-G: 7 breakages of 38 baseline solves against 15.5 expected ($p = 0.003$). We pre-registered a replication of that cell on 80 fresh instances drawn from the issues the matrix had not used, running both arms again rather than reusing a baseline. It did not replicate: 14 breakages of 37 baseline solves, a rate of 0.378 against a resample’s 0.408 ($p = 0.425$). One cell in six at $p = 0.003$ is what multiple comparisons produce, and the two other enhancers paired with

the same solver had already declined to show the effect. We withdraw the observation rather than report it.

This experiment is a small probe (20 issues, seven of them rescuable) and cannot by itself exclude a small positive effect. Section V-G supplies the open-weight arm at scale, and reaches the same null. It does, however, close the most obvious escape route for the RQ1 null: giving the enhancer the codebase does not make enhancement transfer to correctness.

G. Append-only enhancement

The enhancers rewrite rather than augment, so every delta in Table I confounds two changes: context the agent adds, and reporter-written content it deletes. To separate them we repaired all 236 enhancements to be append-only, requiring the original report to survive verbatim with the agent’s findings appended, and re-solved the full three-enhancer by three-solver matrix on 80 issues. That 235 of the 236 needed repair is itself the clearest measure of how these agents behave: replacing the report is not an occasional failure but what they do.

The repair worked as intended. The original text survives verbatim in all 240 enhanced reports, and the agents still add substantially: median output is $2.44\times$ the input for trae, $2.08\times$ for OpenHands and $1.71\times$ for mini-SWE-agent, or roughly one to two kilobytes of added investigation per report.

The effect is absent. Across the six responsive cells (SWE-agent submits a patch on 2–6 of 80 instances, so its zeros carry no information), the arms are $165 \rightarrow 164$. The rescue rate is $59/315 = 0.187$ where a resample of the solver gives 0.199, and the breakage rate is $60/165 = 0.364$ against 0.408. No cell reaches significance on McNemar, and none rescues above chance. Purely additive enhancement is still a resample, so the confound was not concealing a benefit.

H. Executing the fix criterion

A test’s class is a behaviour, not a property of a diff: a fail-to-pass test is one that fails at the base commit with the test patch applied and passes once the gold patch is added. We re-derived both label sets that way. The post-gold half was already available, because the gold-patch probe of Section III ran that exact configuration for all 279 instances and recorded per-test outcomes, so only the pre-gold half needed running.

Execution finds a fail-to-pass test in 85 of 279 instances, against the 225 the offline labels claim, with the two sets agreeing on 45. Rescoring the stored solver patches under the stricter criterion costs nothing further, since the harness records per-test outcomes during scoring.

The criterion change does not fall evenly on the solvers (Table X). Aider’s credited solves are genuine fixes and survive it. Every one of OpenHands’ disappears, which follows from how that solver fails: it terminates without producing a patch, and a repository left untouched keeps the P2P suite green. The solver-capability gap reported in Section IV-A3 is therefore understated rather than overstated.

Enhancement remains null under the stricter criterion. Across 16 comparisons spanning three agentic enhancers, the zero-shot prompt, the reward-gated CL-Enhanced, both responsive solvers, and both Qwen3-32B and GPT-5-mini, the

TABLE X
SAME INSTANCES AND SAME STORED PATCHES, SCORED TWO WAYS.

Solver (4 arms each)	P2P-resolved	Executed F2P
Aider	27	25
OpenHands	15	0

pooled arms are $75 \rightarrow 66$ over 738 paired trials, and no comparison reaches significance. Changing the metric does not change the finding.

I. On Reporting a Negative Result

The design decisions above follow from what it takes to make a null credible. We pre-registered the threshold, the sample and the reading of each outcome before running the two mechanism experiments, which is the practice registered reports were introduced to software engineering to support [29], and we report the outcomes that followed including the one that removed our only positive finding. Paige et al. [30], introducing a special section on negative results, note that such results are systematically under-published and that this costs the community duplicated effort. A study that asks whether a popular intervention works, and finds that it does not, is useful precisely to the extent that it forecloses the question, which requires reporting the failed replication as prominently as the effect it removes.

J. Implications for Practice

For automated repair on already-actionable issues, effort is better spent on the solver and the underlying model than on rewriting the input. Enhancement systems should prioritize verified, factual context injection over generative analysis, and enhancer–solver co-design over generic rewriting. The solving-as-evaluation paradigm gives researchers an objective, reproducible measure of enhancement quality; before deploying enhancement, teams should validate that their specific enhancer–solver combination produces positive deltas on representative, execution-gated data.

VI. THREATS TO VALIDITY

Construct: the metric is P2P-only. Our primary criterion is no-regression on P2P tests, augmented by the applied test patch, rather than the stricter F2P criterion, because the shipped F2P labels were parsed from the test patch rather than executed. Section V-H addresses this directly: we re-derive both label sets from execution and rescore every stored condition, and the finding is unchanged across 16 comparisons. The residual limitation is coverage rather than criterion. Only 85 of 279 instances carry an executed fail-to-pass test, so the strict rescore is well powered against a large or directional effect but cannot exclude a small one on the remaining issues.

One consequence bounds how far the per-model counts in Table IX can be read. Since the criterion requires only that the P2P suite be green, a submission that fails to apply leaves the repository untouched, keeps P2P green, and is therefore credited as a solve. Auditing the submissions confirms this

occurs: all four of Llama-3.3-70B’s baseline solves in Table IX are of this kind: one submission is 60 characters of English prose rather than code, one is a raw Python source file rather than a diff, and two are multi-megabyte `diff -ruN` dumps in which the entire repository appears as deleted (the agent having destroyed its own workspace). In each case the instance carries no F2P tests and the harness’s own report records `resolved: false`. The effect is confined to the low end of the capability range, where models submit malformed output at all, and GPT-5-mini’s submissions are uniformly well-formed, but it means the small nonzero counts among weak models should be read as an upper bound, and it argues for requiring a non-empty, cleanly-applying patch as a precondition for counting a solve in future work.

Construct: enhancement changes two things at once. We measured how much of the original report survives enhancement. Across 236 successful enhancements by three agents (OpenHands, trae, mini-SWE-agent), one preserved the original text verbatim and one retained at least 90% of the original’s substantial lines. Median output length was $1.11\times$ the input for OpenHands, $1.43\times$ for trae and $0.69\times$ for mini-SWE-agent, so the loss is not a matter of summarisation alone: trae returns longer text while still discarding most of the reporter’s lines. In one instance an agent returned 199 characters for a 6,619-character issue, dropping the reproduction steps. Each enhanced condition therefore adds agent-derived context and removes reporter-written content in the same operation, and a null Δ admits two readings the design cannot separate: the added context does not help, or its contribution is offset by what was deleted. The deltas in Table I should be read under this limitation. Constraining enhancement to be append-only isolates the added context. Section V-G reports that variant on the full matrix, and the result is null, so the confound does not account for the deltas in Table I.

Internal: run-to-run solver non-determinism. Baseline and enhanced conditions are separate stochastic runs, so a fraction of the discordant pairs reflects variance rather than enhancement. This is quantifiable in the CL-Enhanced arm: on abstained (text-identical) issues, 6 of 9 SWE-agent “losses” are pure variance, inflating the full-set comparison to a spurious $p = 0.004$ that the enhanced-only subset corrects to a null. Because our headline effects are null or small, this variance widens confidence intervals rather than creating a false positive; a definitive study should run multiple seeds per condition.

Construct: CL-Enhanced operates out of distribution. Its reward model and RAG corpus were trained on an issue-difficulty dataset, not repository-repair issues; applying it here is a transfer test, and its high abstention (57%) is partly an out-of-domain artifact.

External: single model, one collection, execution-gated selection. The controlled matrix uses a single open-weight model (Qwen3-32B). A stronger, function-calling-native model changes absolute rates sharply, and our cross-model check (Section V-D) shows GPT-5-mini raising the OpenHands baseline from 22% to 56%, but *not* the enhancement effect, which remains a directionless null ($\Delta +4$, $p = 0.712$); we nonetheless cannot rule out a different effect for other model families or agent scaffolds. The 382 issues

are, by construction, actionable, so they under-represent the genuinely underspecified reports where enhancement is most plausibly useful. Our dataset is also restricted to popular Python repositories, limiting generalizability.

Conclusion: evaluability coverage. We score only the 73.0% (279/382) gold-validated subset; the un-evaluable 27% are excluded rather than assumed-failed. The multi-method gate is designed to keep that fraction small and repository-diverse.

VII. CONCLUSION

We asked whether improving a bug report’s text improves an LLM agent’s fix, and answered it extrinsically, at scale, by executing patches against real tests. Across 382 issues, a baseline and enhancers spanning a zero-shot prompt to a reward-gated agent, three solvers, and a fixed model, issue enhancement produced no statistically significant improvement in fix correctness (all McNemar $p \geq 0.489$ at $n = 279$ across the controlled agentic enhancers, with a zero-shot prompt enhancer likewise never helping), and the one small-sample signal regressed toward zero under replication. What proved robust is that solver choice governs outcome (Aider 44% $\gg$ OpenHands 22% $\gg$ SWE-agent 6%) and that enhancement perturbs a quarter-to-a-third of outcomes without net direction. The negative result holds for our own selective, reward-gated method as well: on the issues it rewrote it improves no solver, and its most notable behavior is to abstain on the majority of already-actionable reports. No feature predicts a better enhancement, yet enhancement reliably raises the intrinsic quality score, so text-quality gains do not transfer to program repair. We offer this as a rigorous negative result, made trustworthy by a gold-gated, multi-method evaluability protocol (73.0% coverage), an artifact-recovery discipline, and two robustness checks that address the objections a negative result invites: an append-only variant that isolates added context from deleted content, and a fix criterion re-derived from execution rather than from parsing test-patch diffs. Neither changes the conclusion. What the enhanced conditions do to a solver is what running that solver a second time does, including the direction: the weaker solver rises and the stronger one falls, both by margins a resample predicts. Whether enhancement helps on genuinely underspecified reports, and whether a stronger solver model changes the picture, are the natural next studies. We release the benchmark, evaluation pipeline, and experimental artifacts to support future research.

REFERENCES

- [1] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: Can language models resolve real-world GitHub issues?” in *Proc. ICLR*, 2024.
- [2] L. Zhang, S. He, C. Zhang, Y. Kang, B. Li, C. Xie, J. Wang, M. Wang, Y. Huang, S. Fu, E. Nallipogu, Q. Lin, Y. Dang, S. Rajmohan, and D. Zhang, “SWE-bench goes live!” *arXiv preprint arXiv:2505.23419*, 2025.
- [3] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “SWE-agent: Agent-computer interfaces enable automated software engineering,” in *Proc. NeurIPS*, 2024.
- [4] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, H. H. Tran, F. Li, R. Ma, M. Zhang, B. Yu, J. Yang, S. Yao, Z. Liu, and G. Neubig, “OpenHands: An open platform for AI software developers as generalist agents,” *arXiv preprint arXiv:2407.16741*, 2024.

- [5] P. Gauthier, "Aider: AI pair programming in your terminal," 2024. [Online]. Available: <https://aider.chat>
- [6] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, "AutoCodeRover: Autonomous program improvement," in *Proc. ISSTA*, 2024, pp. 1592–1604.
- [7] N. Bettenburg, S. Just, A. Schröter, C. Weiss, R. Premraj, and T. Zimmermann, "What makes a good bug report?" in *Proc. FSE*, 2008, pp. 308–318.
- [8] T. Zimmermann, R. Premraj, N. Bettenburg, S. Just, A. Schröter, and C. Weiss, "What makes a good bug report?" *IEEE Trans. Softw. Eng.*, vol. 36, no. 5, pp. 618–643, 2010.
- [9] C. Sun, D. Lo, S. Khoo, and J. Jiang, "Towards more accurate retrieval of duplicate bug reports," in *Proc. ASE*, 2011, pp. 253–262.
- [10] A. Lazar, S. Ritchey, and B. Sharif, "Generating duplicate bug report datasets," in *Proc. MSR*, 2014, pp. 392–395.
- [11] J. Anvik, L. Hiew, and G. C. Murphy, "Who should fix this bug?" in *Proc. ICSE*, 2006, pp. 361–370.
- [12] G. C. Murphy and D. Cubranic, "Automatic bug triage using text categorization," in *Proc. SEKE*, 2004, pp. 92–97.
- [13] C. P. Wong, Y. Xiong, H. Zhang, D. Hao, L. Zhang, and H. Mei, "Boosting bug-report-oriented fault localization with segmentation and stack-trace analysis," in *Proc. ICSME*, 2014, pp. 181–190.
- [14] L. Moreno, J. J. Treadway, A. Marcus, and W. Shen, "On the use of stack traces to improve text retrieval-based bug localization," in *Proc. ICSME*, 2015, pp. 151–160.
- [15] R. Aleithan, H. Xue, M. M. Mohajer, E. Nnorom, G. Uddin, and S. Wang, "SWE-Bench+: Enhanced coding benchmark for LLMs," 2024, arXiv:2410.06992.
- [16] OpenAI, "Introducing SWE-bench Verified," 2024. [Online]. Available: <https://openai.com/index/introducing-swe-bench-verified/>
- [17] X. Deng, J. Da, E. Pan, Y. Y. He, C. Ide, *et al.*, "SWE-Bench Pro: Can AI agents solve long-horizon software engineering tasks?" 2025, arXiv:2509.16941.
- [18] Z. Jiang, D. Lo, Z. Liu, *et al.*, "Agentic software issue resolution with large language models: A survey," 2025, arXiv:2512.22256.
- [19] J. Acharya and G. Ginde, "Can we enhance bug report quality using LLMs? An empirical study of LLM-based bug report generation," in *Proc. EASE*, 2025, arXiv:2504.18804.
- [20] S. M. F. Al Fahim, M. N. Rafi, Z. Ma, D. J. Kim, and T.-H. Chen, "Crash report enhancement with large language models: An empirical study," 2025, arXiv:2509.13535.
- [21] A. Paleyes, R. Sendyka, D. Robinson, C. Cabrera, and N. D. Lawrence, "Code roulette: How prompt variability affects LLM code generation," in *Proc. LLM4Code @ ICSE*, 2026, arXiv:2506.10204.
- [22] H. Femmer, D. Méndez Fernández, S. Wagner, and S. Eder, "Rapid quality assurance with requirements smells," *J. Syst. Softw.*, 2017, arXiv:1611.08847.
- [23] Z. Fei, J. Ge, C. Li, T. Wang, Y. Li, H. Zhang, L. Huang, and B. Luo, "Patch correctness assessment: A survey," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 2, 2025, doi:10.1145/3702972.
- [24] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, "An empirical evaluation of using large language models for automated unit test generation," *IEEE Trans. Softw. Eng.*, vol. 50, no. 1, pp. 85–105, 2024.
- [25] X. Zhou, M. Weyssow, R. Widyasari, T. Zhang, J. He, Y. Lyu, J. Chang, B. Zhang, D. Huang, and D. Lo, "LessLeak-Bench: A first investigation of data leakage in LLMs across 83 software engineering benchmarks," 2025, arXiv:2502.06215.
- [26] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Agentless: Demystifying LLM-based software engineering agents," 2024, arXiv:2407.01489.
- [27] Y. Ma, Y. Li, Y. Dong, X. Jiang, R. Cao, J. Chen, F. Huang, and B. Li, "Alibaba LingmaAgent: Improving automated issue resolution via comprehensive repository exploration," in *Proc. FSE Companion*, 2025, arXiv:2406.01422.
- [28] A. Arcuri and L. Briand, "A Hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering," *Softw. Test. Verif. Reliab.*, vol. 24, no. 3, pp. 219–250, 2014.
- [29] N. A. Ernst and M. T. Baldassarre, "Registered reports in software engineering," *Empir. Softw. Eng.*, 2023, arXiv:2302.03649.
- [30] R. F. Paige, J. Cabot, and N. A. Ernst, "Foreword to the special section on negative results in software engineering," *Empir. Softw. Eng.*, vol. 22, pp. 2453–2456, 2017.
- [31] P. Fathollahzadeh and Y. Zou, "Replication package: Benchmarking LLM-based agents for GitHub issue enhancement," 2026. [Online]. Available: <https://github.com/pouyafath/BenchmarkLLMAgent>